

Practica 5: Agrupamiento y Clasificación

Dra. María Elena Martínez Perez
M. en C. Miguel Ángel Veloz Lucas

27 de mayo de 2026

1 Reglas generales para el desarrollo de las Prácticas de Laboratorio

- Deberás respetar la estructura general de este documento, i.e, entregar tu práctica con las secciones: objetivos, introducción, desarrollo, código, conclusiones y referencias.
- El desarrollo de la práctica deberá ser auténtico. Aquellos equipos que presenten los mismos cálculos, código fuente, etcétera, serán sancionados.
- El día de entrega establecido deberá ser respetado por todos los equipos. La hora límite de entrega es la hora de clase y no se reciben trabajos que sean terminados durante la hora de clase.
- Deberás entregar el documento en el classroom antes de la fecha límite.

2 Objetivos

Esta práctica introduce el modelo de bolsa de palabras o *Bag of Visual Words* (BoVW) para la clasificación de imágenes. El flujo de trabajo incluye:

- Extraer descriptores SIFT de un conjunto de imágenes.
- Construir un vocabulario visual aplicando k-means.
- Representar cada imagen mediante un histograma normalizado de palabras visuales.
- Clasificar las imágenes.
- Evaluar el modelo con exactitud y matriz de confusión.
- Experimentar con distintos parámetros (número de palabras, tipo de clasificador, etc.).

3 Introducción

El reconocimiento de patrones en imágenes es un área fundamental de la visión por computadora. Uno de los enfoques más populares para la clasificación de escenas u objetos es el modelo *Bag of Words* (BoVW), inspirado en el procesamiento de texto. En lugar de palabras, se utilizan *palabras visuales* obtenidas a partir de descriptores locales de las imágenes. Este método es robusto frente a variaciones de escala, rotación e iluminación parcial.

3.1 Bolsa de palabras o Bag of Words en texto

El modelo (BoW) es ampliamente utilizado en procesamiento de lenguaje natural. Consiste en representar un documento como un vector de frecuencias de palabras de un vocabulario predefinido, ignorando el orden de las palabras pero preservando la información semántica relevante. Por ejemplo, si el vocabulario es $\{\textit{perro}, \textit{gato}, \textit{ratón}\}$, un documento con las palabras “gato gato perro” se representa como $[1, 2, 0]$.

La clave del éxito de BoW es que palabras similares tienden a aparecer en contextos similares, y dos documentos con distribuciones de palabras parecidas probablemente tratan sobre el mismo tema.

3.2 Traslación al dominio visual: palabras visuales

De manera análoga, en el enfoque BoW se considera que una imagen es un “documento” compuesto por “palabras visuales”. Estas palabras no son objetos completos, sino **parches locales de la imagen** que han sido cuantificados en un vocabulario. El proceso típico consta de tres fases:

1. **Extracción de descriptores locales:** Se detectan regiones de interés (por ejemplo, esquinas, bordes, texturas) y se calcula un descriptor numérico que resume la apariencia de cada región. Uno de los descriptores más potentes y ampliamente usados es **SIFT** (Scale-Invariant Feature Transform), desarrollado por David Lowe en 2004. SIFT es invariante a rotación y escala parcial, y robusto a cambios de iluminación. Cada descriptor es un vector de 128 números reales.
2. **Construcción del vocabulario visual:** Se toman todos los descriptores extraídos de un conjunto de entrenamiento y se agrupan mediante un algoritmo de clustering, típicamente *k-means*. Cada centroide resultante define una “palabra visual”. El número k de palabras es un hiperparámetro crítico: valores pequeños producen palabras muy generales, valores grandes producen palabras muy específicas (riesgo de sobreajuste).
3. **Representación de la imagen como histograma:** Para una imagen nueva, se extraen sus descriptores SIFT y cada uno se asigna a la palabra visual más cercana (en distancia euclídea). Se cuenta cuántas veces

aparece cada palabra y se normaliza el histograma para que sume 1. Este histograma es el vector de características que se usará para la clasificación.

3.3 Ventajas del modelo BoW

- **Invariancia parcial:** Al depender de descriptores locales invariantes (SIFT), el modelo tolera cambios de escala, rotación y traslación de los objetos.
- **Compacidad:** Una imagen se reduce de miles de píxeles a un histograma de dimensión k (típicamente entre 100 y 1000), lo que facilita el entrenamiento de clasificadores.
- **Simplicidad conceptual:** La analogía con el texto es fácil de entender y extender.
- **Buen rendimiento:** En su época, BoW logró resultados competitivos en benchmarks como Caltech-101.

3.4 Clasificador empleado: Máquina de Vectores Soporte (SVM)

Una vez que tenemos los histogramas, necesitamos un clasificador que aprenda a separar las distintas categorías. Utilizaremos una **Máquina de Vectores Soporte** con kernel lineal. El SVM busca el hiperplano que maximiza el margen entre las clases. Es especialmente eficaz cuando el número de características es moderado (nuestro k) y hay un número suficiente de ejemplos. La elección del kernel lineal evita la complejidad de kernels no lineales y suele funcionar bien para histogramas normalizados.

3.5 Métricas de evaluación

Para medir el rendimiento del clasificador usaremos dos herramientas estándar:

- **Accuracy** (exactitud): Proporción de aciertos global. Se define como $\frac{TP+TN}{TP+TN+FP+FN}$, aunque en clasificación multiclase se calcula como el número de predicciones correctas dividido entre el total. Es una medida intuitiva pero puede ser engañosa si las clases están desbalanceadas.
- **Matriz de confusión:** Una tabla de tamaño $C \times C$ (donde C es el número de clases) donde la entrada (i, j) indica cuántas muestras de la clase real i fueron clasificadas como clase j . Permite identificar confusiones específicas: por ejemplo, que muchos aviones sean etiquetados como pájaros, o que los coches y las motos se confundan entre sí.

4 Desarrollo

Ejercicio: Implementación del modelo de bolsa de palabras (BoVW) y comparar resultados. Selecciona uno de los tres con-

juntos de datos para la implementación del modelo variando la configuración de sus hiperparámetros. Se sugiere utilizar un subconjunto de **Caltech-101** (disponible en `torchvision`) con tres categorías:

- `airplane` (avión).
- `car_side` (coche de perfil).
- `motorbikes` (motocicletas).

Alternativamente, para probar con más clases o imágenes propias:

- **CIFAR-10**: 10 clases de objetos pequeños (requiere redimensionar a 128×128).
- **Oxford Flowers-102**: 102 tipos de flores.
- **Dataset propio**: imágenes organizadas en carpetas por categoría.

1. El flujo de trabajo implementado en el script es el siguiente:

- (a) Descargar y filtrar el dataset (3 clases, imágenes redimensionadas a 128×128).

Se utilizó el subconjunto sugerido de **Caltech-101** con las tres categorías `airplanes`, `car_side` y `Motorbikes`. La **descarga** se realiza con la función `descargar_caltech()`, que baja el archivo `caltech-101.zip` (≈ 137 MB) desde el mirror oficial de Caltech y lo cachea en disco para no repetir la descarga. El **filtrado** lo hace `extraer_clases()`, que abre el `tar.gz` anidado dentro del zip y extrae únicamente los `.jpg` de las tres clases de interés. Finalmente, `cargar_imagenes()` construye un conjunto **balanceado** tomando **120 imágenes por clase** (360 en total), todas convertidas a escala de grises y redimensionadas a 128×128 píxeles. Se fijó la semilla `SEED = 42` para garantizar la reproducibilidad y el tamaño de vocabulario por defecto fue $k = 200$. La Figura 1 muestra cinco ejemplos por clase.



Figure 1: Muestra de imágenes por clase (128×128 , escala de grises).

(b) Extraer descriptores SIFT de todas las imágenes.

Con `cv2.SIFT_create()` se aplicó `detectAndCompute` a cada imagen, obteniendo sus *keypoints* y un descriptor de **128 dimensiones** por cada uno. El número de descriptores por imagen es variable: **mínimo 29, máximo 296 y una media de 121**, sin ninguna imagen vacía. La Figura 2 muestra los *keypoints* detectados sobre una imagen de cada clase; el tamaño y la orientación de cada círculo reflejan la escala y la rotación del punto de interés.

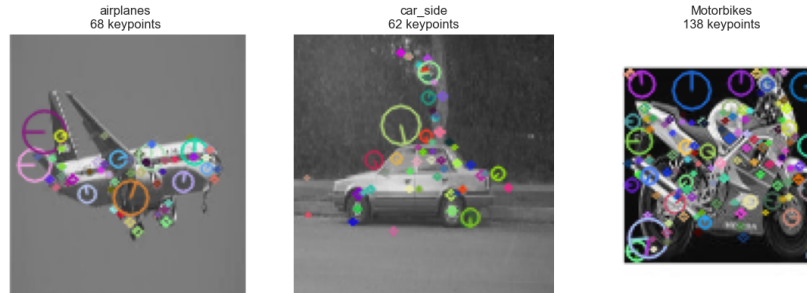


Figure 2: Keypoints SIFT detectados sobre una imagen representativa de cada clase.

(c) Ejecutar k-means con k clusters para obtener el vocabulario.

Todos los descriptores se apilaron en una única matriz de tamaño 43612×128 y se agruparon con **k-means** ($k = 200$, `n_init = 3`, quedándose con la mejor de 3 inicializaciones). Cada uno de los 200 centroides resultantes constituye una **palabra visual**, de modo que

el vocabulario visual es una matriz 200×128 . El entrenamiento de k-means tardó aproximadamente 6.3 s.

- (d) Para cada imagen, construir su histograma de palabras visuales (Mostrar 5 de cada agrupación).

Cada descriptor SIFT de una imagen se asigna a la palabra visual más cercana y se cuenta su frecuencia, generando un histograma de dimensión $k = 200$ **normalizado en L1** (suma 1). La matriz de características resultante tiene forma 360×200 . La Figura 3 muestra cinco histogramas por clase: se aprecia que las imágenes de una **misma clase** producen distribuciones de palabras visuales **parecidas entre sí**, lo que es la base del poder discriminativo del modelo.



Figure 3: Histogramas BoVW: 5 ejemplos por clase (eje x = palabra visual, eje y = frecuencia normalizada).

- (e) Dividir el conjunto en entrenamiento (80%) y prueba (20%).
El conjunto se dividió de forma **estratificada** (manteniendo la proporción de clases) en 80% de entrenamiento (**288 imágenes**, 96 por clase) y 20% de prueba (**72 imágenes**, 24 por clase).
- (f) Entrenar un SVM lineal con los histogramas de entrenamiento utilizando búsqueda de hiperparámetros (*grid search*).
Se entrenó un **SVM con kernel lineal**, ajustando el parámetro de regularización C mediante *grid search* con validación cruzada de 5 particiones sobre la rejilla $C \in \{0.01, 0.1, 1, 10, 100\}$. El mejor valor encontrado fue $C = 100$, con una exactitud media en validación cruzada (sobre el conjunto de entrenamiento) de 0.8403.
- (g) Predecir las etiquetas del conjunto de prueba y calcular exactitud, comenta tus resultados.

Sobre el conjunto de prueba (datos no vistos) el modelo alcanzó una **exactitud de 0.8472** ($\approx 85\%$). El reporte de clasificación por clase se muestra en la Tabla 1.

Clase	Precisión	Recall	F1-score
airplanes	0.84	0.88	0.86
car_side	0.79	0.96	0.87
Motorbikes	0.94	0.71	0.81
Exactitud global	0.85 (72 imágenes)		

Table 1: Métricas por clase del SVM lineal ($k = 200$) sobre el conjunto de prueba.

El modelo BoVW + SVM lineal alcanza una exactitud cercana al 85%, muy por encima del azar (33% para 3 clases balanceadas). La clase `car_side` es la que mejor se recupera (*recall* de 0.96), mientras que `Motorbikes` presenta una precisión muy alta (0.94) pero un *recall* más bajo (0.71): el clasificador rara vez confunde otra clase con una moto, pero sí deja escapar algunas motos hacia las otras categorías.

(h) Mostrar la matriz de confusión, comenta tus resultados.

La Figura 4 muestra la matriz de confusión. La diagonal concentra la gran mayoría de las predicciones, lo que confirma la buena separación de las clases. Los pocos errores se reparten principalmente entre `airplanes` y `Motorbikes`, clases que comparten estructuras locales similares (bordes alargados, fondos uniformes), mientras que `car_side` resulta la categoría más fácilmente separable.

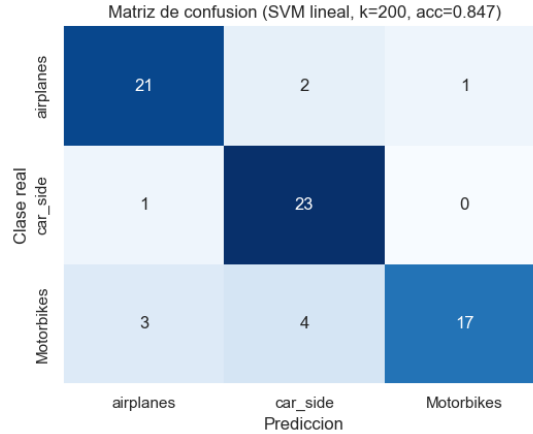


Figure 4: Matriz de confusión del SVM lineal ($k = 200$, exactitud = 0.847).

2. Evaluación y comparación de resultados:

- (a) **Variar el número de palabras visuales** ($k = 50, 200, 500$). ¿Cómo afecta la exactitud y el tiempo de cómputo?

Reutilizando la misma partición de entrenamiento/prueba para una comparación justa, se repitió todo el proceso con tres tamaños de vocabulario. Los resultados se resumen en la Tabla 2 y la Figura 5.

k	Exactitud	Mejor C	$t_{k\text{-means}}$	t_{total}
50	0.7500	10	2.2 s	2.3 s
200	0.8472	100	5.9 s	6.0 s
500	0.8333	100	12.6 s	12.7 s

Table 2: Exactitud y tiempo de cómputo en función del tamaño del vocabulario k .

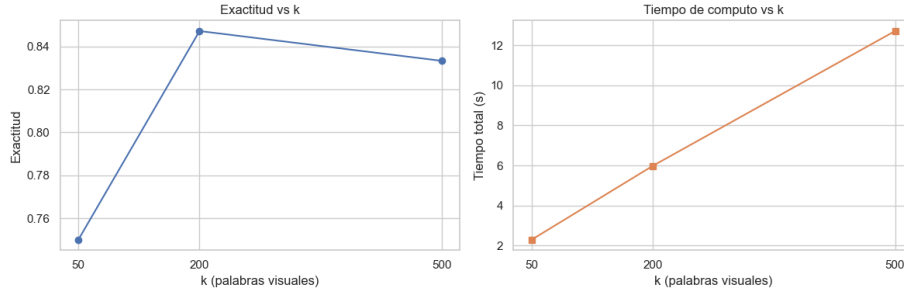


Figure 5: Exactitud (izquierda) y tiempo total de cómputo (derecha) frente a k .

Al aumentar k el vocabulario es más expresivo y la exactitud crece de forma notable de $k = 50$ (0.75) a $k = 200$ (0.85); sin embargo, al pasar a $k = 500$ la exactitud deja de mejorar e incluso baja ligeramente (0.83), señal de que los histogramas se vuelven muy dispersos y empieza a aparecer sobreajuste. El tiempo de cómputo, en cambio, crece de manera marcada (sobre todo el de k-means, que casi se duplica en cada salto). El valor $k \approx 200$ ofrece el mejor compromiso entre exactitud y costo.

- (b) **Cambiar el clasificador:** sustituir SVM por Random Forest y comparar resultados.

Con el mismo vocabulario ($k = 200$) se entrenó un Random Forest ajustando por *grid search* el número de árboles y la profundidad máxima ($n_estimators \in \{100, 300\}$, $max_depth \in \{None, 20\}$).

Los mejores parámetros fueron `n_estimators = 300` y `max_depth = None`. La comparación se resume en la Tabla 3 y la Figura 6.

Clasificador	Exactitud (prueba)
SVM lineal	0.8472
Random Forest	0.8333

Table 3: Comparación de clasificadores con $k = 200$.

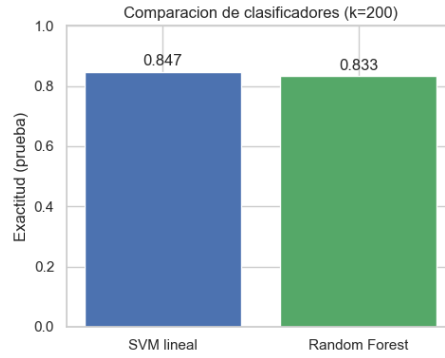


Figure 6: Comparación de exactitud entre SVM lineal y Random Forest ($k = 200$).

Con el mismo vocabulario, el SVM lineal (0.847) y el Random Forest (0.833) ofrecen resultados muy comparables; la diferencia es de apenas una imagen. El SVM lineal resulta ligeramente superior y más económico de entrenar, lo cual es coherente con que los histogramas normalizados son representaciones de dimensión moderada sobre las que un separador lineal funciona muy bien.

- (c) **Agregar validación cruzada:** hacer validación cruzada con 5 particiones e informar la media y desviación estándar de la exactitud.

Para obtener una estimación más fiable que la de una única partición 80/20, se realizó validación cruzada estratificada de 5 particiones sobre todo el conjunto, empleando los mejores hiperparámetros hallados para cada modelo. Los resultados se muestran en la Tabla 4 y la Figura 7.

Clasificador	Media	Desviación estándar
SVM lineal	0.8722	0.0297
Random Forest	0.8833	0.0286

Table 4: Exactitud en validación cruzada de 5 particiones.

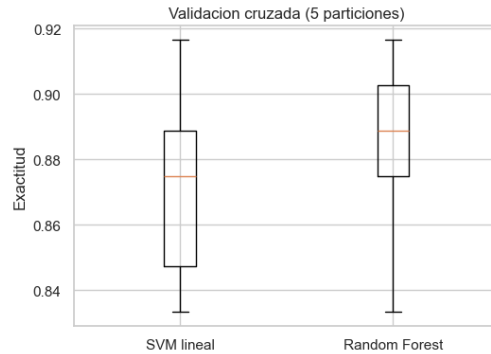


Figure 7: Distribución de la exactitud en las 5 particiones (boxplot).

Con 5 particiones la exactitud media se sitúa en torno a 0.87–0.88 para ambos clasificadores, con una **desviación estándar pequeña** (≈ 0.03), lo que indica que el modelo es **estable** y que el resultado de la partición única (inciso g) no fue fortuito. Ambos clasificadores son estadísticamente equivalentes: sus intervalos se solapan ampliamente.

3. Opcional

- (a) **Utilizar una base de datos diferente** y repetir la implementación del modelo adaptando la carga de datos.

4. Preguntas:

- ¿Por qué es necesario normalizar los histogramas?
 Porque cada imagen genera un número distinto de descriptores SIFT (depende de su tamaño, textura y cantidad de keypoints detectados). Si no se normaliza, una imagen con muchos keypoints tendría conteos absolutos mayores y el clasificador compararía magnitudes en lugar de la distribución de palabras visuales. Al normalizar (L1, que el histograma suma 1) el vector representa proporciones de cada palabra visual, lo que lo hace invariante al número total de descriptores y pone todas las imágenes en una escala comparable; esto además favorece el buen comportamiento del SVM lineal.
- ¿Qué ocurre si k es demasiado pequeño? ¿y si es demasiado grande?
 Con k demasiado pequeño, las palabras visuales son muy genéricas: estructuras locales distintas caen en la misma palabra, los histogramas pierden poder discriminativo y el modelo subajusta (las clases se confunden). Con k demasiado grande, las palabras son muy específicas: los histogramas se vuelven muy dispersos sparse), aparece riesgo de sobreajuste, se necesita más cantidad de datos y crece mucho el costo de k-means y de la asignación; la exactitud se estanca

o incluso baja (como se observa en el inciso 2(a)). Existe un valor intermedio óptimo (≈ 200 en este caso).

- ¿Podría utilizarse este método para detección de objetos en lugar de clasificación?

Sí, pero no directamente. La clasificación BoVW describe toda la imagen con un único histograma y por tanto no localiza el objeto. Para detección habría que aplicar BoVW sobre regiones candidatas y clasificar cada región, o incorporar información espacial.

5 Código

El código fuente completo de la práctica se encuentra implementado en el cuaderno de Jupyter `Practica_5_Agrupamiento_y_Clasificacion.ipynb`.

6 Conclusiones

Se implementó por completo el modelo de **Bolsa de Palabras Visuales (BoVW)** para clasificar imágenes de 3 categorías de Caltech-101 (`airplanes`, `car_side`, `Motorbikes`). El flujo SIFT, k-means (vocabulario visual), histograma normalizado, clasificador alcanza una exactitud de $\approx 85\%$ en prueba y ≈ 0.87 en validación cruzada de 5 particiones, muy por encima del azar (33%).

Las principales conclusiones son las siguientes:

- El **tamaño del vocabulario** k es el hiperparámetro más influyente: la exactitud crece con rendimientos decrecientes y el tiempo de cómputo aumenta de forma marcada; $k \approx 200$ fue el mejor compromiso.
- La **normalización de los histogramas** es esencial para que la representación sea invariante al número de descriptores.
- **SVM lineal** y **Random Forest** dan resultados comparables; la validación cruzada confirma que el modelo es estable (baja desviación estándar).
- Los errores se concentran entre clases que comparten apariencia local (`airplanes` vs `Motorbikes`), mientras que `car_side` se separa con facilidad.
- El pipeline es **genérico y reutilizable**: aplicarlo a otra base de datos solo exige cambiar la carga de datos.

7 Referencias

- D. G. Lowe, "Distinctive image features from scale-invariant keypoints", IJCV 2004.
- G. Csurka et al., "Visual categorization with bags of keypoints", ECCV 2004.
- Scikit-learn documentation: <https://scikit-learn.org>
- OpenCV Python tutorials: <https://docs.opencv.org>